

コンテキスト・エンジニアリング (Context Engineering)

I. Prompt Engineering と Context Engineering

II. Context Engineeringの進化

III. 実装戦略とエントロピー削減

IV. システム設計の三本柱

V. 未来ビジョン: Semantic OS

# I. Prompt Engineering と Context Engineering

## Prompt Engineering (どう質問するか)

エンドユーザー向けの手法

目的: できるだけ明確に要求を伝えること

## Context Engineering (何を与えるか)

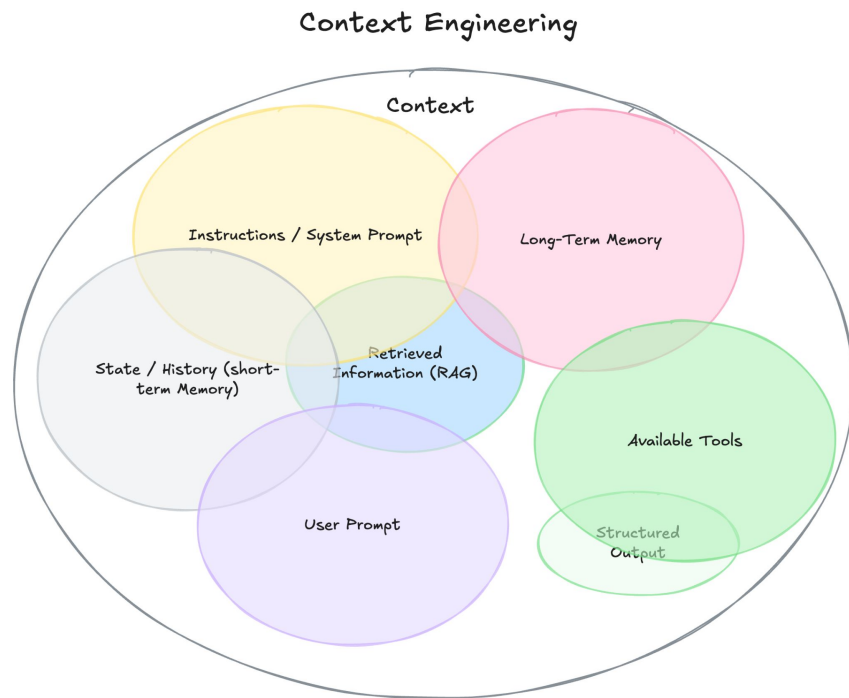
開発者向けの設計思想

### 任務:

モデルに入力する前に、

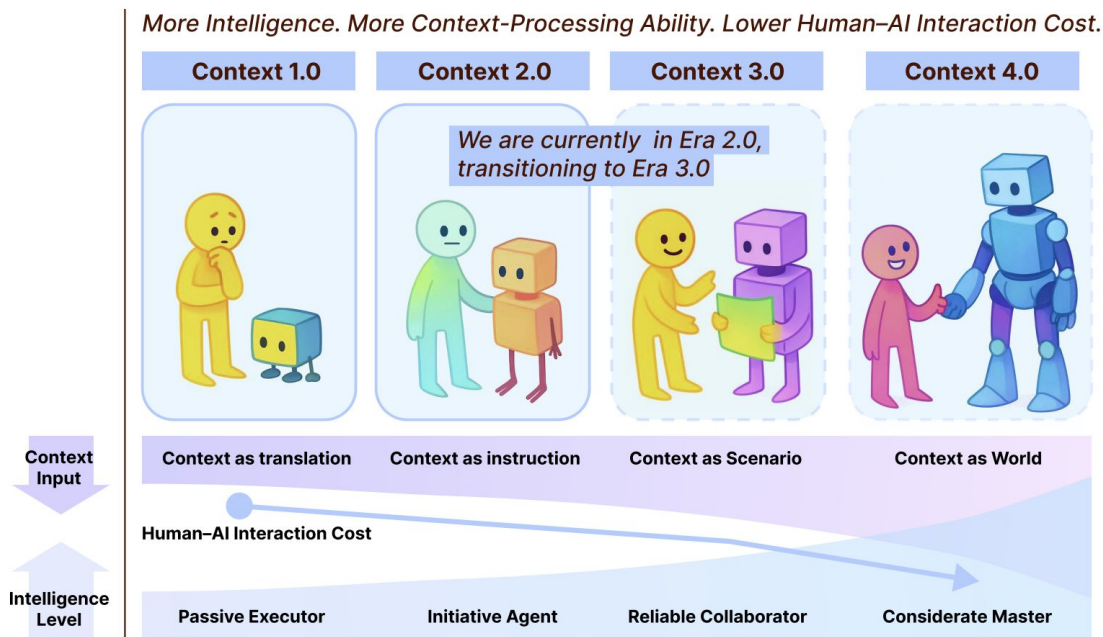
RAG、対話履歴、例などの背景情報を

動的に組み立てるシステムを構築すること。



## II. Context Engineeringの進化

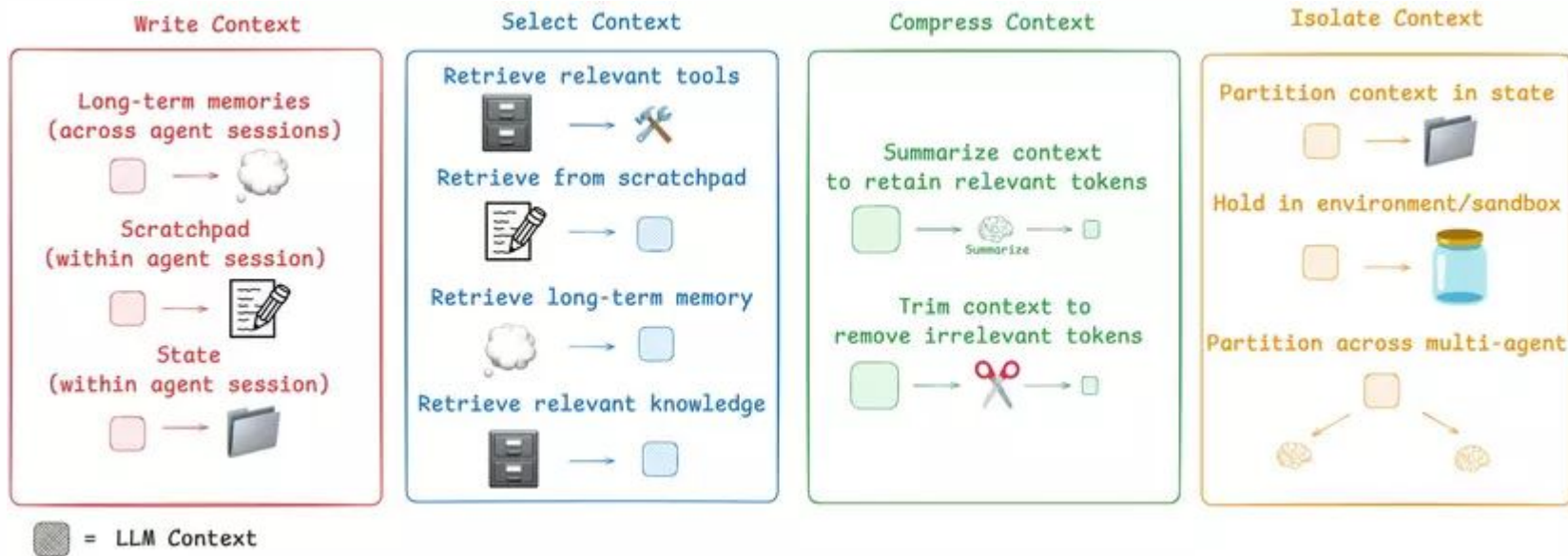
論文「Context Engineering 2.0: The Context of Context Engineering」では、20年以上にわたる発展が4つの時代に分けられると述べられている。



人間の高エントロピー（曖昧・非構造）な意図を LLMが処理できる低エントロピー（構造化）表現へ変換する。

### III. 実装戦略とエントロピー削減

Langflowの公式サイトにあるコンテキストに関する技術ブログでは、ある一つの戦略が紹介されています。



## IV. システム設計の3つの柱

### 1. 収集と保存 (Collection & Storage)

データの出どころを決める

保存形式を決める

コンテキストのライフサイクルを設計する

### 2. 管理 (Management)

画像・音声・テキストなど多様な形式を扱う

メモリを分けて管理する (作業用・長期・セグメント)

エージェント間でコンテキストを分離し、安全性を確保する

### 3. 使用 (Usage)

コンテキストを共有する

能動的に推論する (proactive inference)

エージェントのライフサイクル全体でコンテキストを維持する

## V. 未来のビジョン セマンティックOS (Semantic OS)

コンテキストを、OSにおけるRAM管理のような「第一級の資源」として扱う。

### 主な方向性

#### 大規模意味保存

ナレッジグラフ  
非可逆ない圧縮

#### 説明可能な推論

意思決定の追跡  
人間による制御と修正

#### 生涯(しょうがい)コンテキスト

長年にわたりユーザーと共に進化  
プライバシー保護を確保